



Machine Learning for Petroleum Engineers Using Python

Módulo 7 · Clase 2

El Tasador — Un Modelo de Punta a Punta



Carlos Enrique Mosquera Trujillo



SLB Ecuador

Dónde Estamos: Hoy se Paga lo Pendiente

✓ De la Clase 1 ya tienen:

- ✓ Los **cuatro fundamentos** de un encargo: objetivo, contexto, restricciones, criterio de aceptación
- ✓ Gráficas que se defienden solas
- ✓ La regla del cero: un cero perfecto casi siempre es una **ausencia**

Y quedó pendiente la app con link — el tiempo no alcanzó. Hoy se paga, y con intereses.

Recordatorio del proyecto: equipos de hasta 3, entrega hasta el jueves de la próxima semana. Lo de hoy es exactamente lo que su proyecto necesita.

▶▶ Hoy:

- **Un modelo de punta a punta**, con un solo caso de inicio a fin: el tasador de diamantes
- Certificar → entrenar → examinar → guardar → **servir** → blindar
- ★ Y el mismo flujo, **hecho por ustedes** en otro caso — que es la pieza central de su proyecto

El Caso · El Comprador de Diamantes

EL PROBLEMA

Su cliente compra lotes de piedras. Hoy tasa a **ojo y por comparables**, piedra por piedra: lento, y depende de quién tasa ese día.

EL COSTO DEL ERROR

Comprar caro **no se nota hasta vender**. Y pasar de largo una piedra buena es margen regalado al competidor.

LA DECISIÓN

Frente a cada piedra: **comprar o pasar**, y a qué precio máximo. Cientos de veces por lote.

QUÉ NOS PIDIÓ

«Un **tasador**: le meto las medidas de la piedra y me dice cuánto vale. Que lo use mi gente en la mesa de compra, sin llamarlos a ustedes.»

Tiene 53.940 compras históricas con precio pagado. Eso es todo lo que hace falta para intentarlo.

Ustedes Ya Hacen Esto

¿Cómo tasa hoy el comprador, sin modelo? **Por comparables:** busca en su memoria piedras parecidas que ya compró, y ajusta por las diferencias.

Eso es *exactamente* lo que hicieron en el Módulo 5 con las reservas: los **análogos** — «de los campos que estuvieron donde está el mío, ¿dónde terminaron?».

Un modelo de precios es la memoria de comparables del comprador — pero con 53.940 casos en vez de los cientos que caben en una cabeza, y disponible para todo su equipo a la vez.

No estamos reemplazando su criterio: estamos poniendo su manera de pensar dentro de una herramienta que no se cansa ni se enferma.

El Mapa de la Clase: Seis Pasos, Un Flujo



Este es **el flujo completo de un producto de datos** — el mismo en cualquier empresa, con cualquier herramienta. Hoy lo recorremos entero una vez conmigo, y una vez ustedes.

Fíjense dónde empieza: no en el modelo. En preguntarle al dato si dice la verdad. Eso ya lo saben hacer desde la Clase 1.

Paso 1 • Certificar

Nadie entrena con datos que no revisó

12 – 28 min

Recordaris de la Clase 1: el Cero que No Es Cero

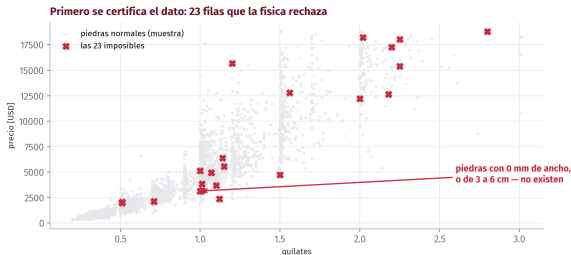
En la Clase 1 vimos 1.812 viajes en efectivo con **exactamente cero** propinas — y la conclusión fácil era falsa: el taxímetro no registra la propina en efectivo. Un cero que era una **ausencia**.

La regla que salió de ahí: en datos reales, **nada es exactamente cero por casualidad**. Y hoy la aplicamos antes de entrenar nada.

```
Voy a entrenar un modelo de precios con el DataFrame 'diamantes' (53.940
filas). Columnas: carat, cut, color, clarity, price [USD], y x, y, z que son
las dimensiones físicas de la piedra en milímetros.
ANTES de cualquier modelo: revisión de calidad. - mínimo, máximo y ceros de cada
columna numérica - dime si algún valor es FÍSICAMENTE imposible para un diamante
(pista: son milímetros) - cuántas filas afectadas, y muéstramelas
```

El mismo prompt de revisión de la Clase 1, afinado al caso. Ya es de ustedes: se corre antes de cada análisis, siempre.

Lo que Aparece: 23 Piedras que No Existen



Piedras con **0 mm** de alguna dimensión — no existen — y un par con 3 a 6 **centímetros**, que serían piedras de museo a precio de anillo. Errores de captura, no diamantes.

La Decisión de Certificación

¿Qué se hace con 23 filas malas entre 53.940? La decisión tiene tres partes, y las tres se **escriben**:

1. **Se rechazan** las 23 — con argumento físico, no estético: una dimensión de 0 mm no es una piedra, es un campo vacío disfrazado de número.
2. **Se le avisa al dueño del dato**, en dos frases: qué se encontró y qué se hizo. Hoy son 23 inofensivas; mañana pueden ser las 5.000 que cambian el negocio.
3. **Queda registrado en el código**: el filtro con su comentario. El que herede esto sabrá qué se quitó y por qué.

Quedan 53.917 piedras certificadas. Recién ahora se puede entrenar.

Pasos 2 y 3 · Entrenar y Examinar

El bosque aprende, y rinde
examen con piedras que no vio

28 – 50 min

Recordaris: Qué Es Entrenar un Modelo

Lo vieron en el Módulo 5, y como acá nada se da por sabido, en una frase:

Entrenar es mostrarle al modelo miles de casos que ya terminaron — piedra, medidas, y lo que de verdad se pagó — hasta que aprende la relación entre las medidas y el precio.

Usamos el **Random Forest** — el bosque de las clases del Módulo 5: cien árboles de decisión, cada uno aprende de una muestra distinta, y la tasación es el voto de todos.

Y la regla de oro del examen, que también ya conocen: se **reserva el 20 %** de las piedras *antes* de entrenar. El modelo jamás las ve. Con esas se le toma el examen — porque examinarlo con piedras que memorizó es hacerle trampa a favor. *¿Por qué no un modelo más moderno y complicado? Porque primero se mide si el simple alcanza. Es la regla del récord a batir, de todo el diplomado.*

Desde Cero: el Modelo No Come Texto

El corte de una piedra es "Ideal" o "Premium" — palabras. Y un modelo solo sabe de números. La conversión estándar se llama `get_dummies`: cada categoría se vuelve una columna de unos y ceros.

```
pd.get_dummies(d[["carat", "cut"]], columns=["cut"])  
# carat  cut_Fair  cut_Good  cut_Ideal  cut_Premium ...  
#   0.23         0         0         1         0  
#   0.21         0         0         0         1
```

La piedra Ideal tiene un 1 en su columna y 0 en las demás. Nada más que eso.

GUARDEN ESTE DETALLE

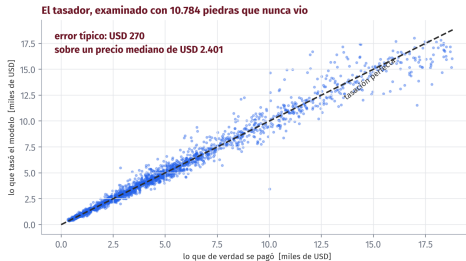
Esta conversión crea **muchas columnas nuevas, en un orden**. La app va a tener que reproducir *exactamente* las mismas columnas en *exactamente* el mismo orden. Vuelve en un rato — y muerde.

El Encargo de Entrenamiento

Con el DataFrame 'd' ya certificado (53.917 piedras):
OBJETIVO: un modelo que estime el precio [USD] a partir de carat, cut, color, clarity, x, y, z, depth y table.
RESTRICCIONES: - RandomForestRegressor de scikit-learn, 100 árboles,
random_state = 0 - reservael20(train_test_split, random_state = 0) -
las columnas de texto (cut, color, clarity) conviértelas en get_dummies
CRITERIO DE ACEPTACIÓN: - repórtame el error en DÓLARES (MAE), no solo el R2: el cliente piensa en dólares - y compáralo contra el precio mediano, para saber si es mucho o poco

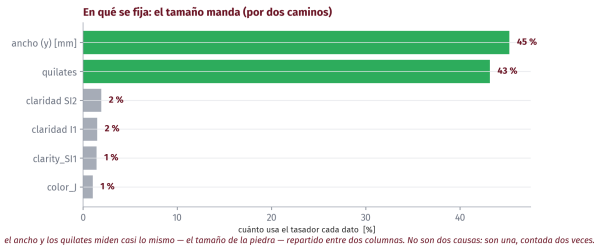
*Fíjense en el criterio: pedir el error **en la unidad del negocio**. Un R^2 no se puede negociar con un vendedor de piedras; «me equivocó 270 dólares típicamente», sí.*

El Examen: 10.784 Piedras que Nunca Vio



Entrenó en **2 segundos**. Error típico: **USD 270** sobre un precio mediano de **USD 2.401** — alrededor de un 11 %. Para una mesa de compra que hoy tasa a ojo, eso ya es una herramienta.

En Qué Se Fija — y la Lectura Honesta



El ancho (45 %) y los quilates (43 %) dominan — y **miden casi lo mismo**: el tamaño, contado dos veces. Estas barras dicen en qué se apoyó el modelo, **no qué causa el precio**. La distinción de siempre.

El Error Silencioso: el Orden de las Columnas

El error más caro de servir modelos no lanza ningún mensaje. Es este: el modelo se entrenó con las columnas en un orden, y la app le arma la fila **en otro**.

SI PASA UNA TABLA CON NOMBRES

scikit-learn compara los nombres de las columnas contra los del entrenamiento y **reclama** si no cuadran. Protección gratis.

SI PASA NÚMEROS PELADOS

Nadie revisa nada: los quilates caen donde iba la profundidad, y la app tasa **basura con cara de precio**. Sin error, sin aviso.

La versión general: el mundo del entrenamiento y el mundo de la app tienen que ser idénticos — mismas columnas, mismo orden, misma preparación. Cuando difieren, el modelo que daba 98 en el cuaderno da basura en producción, y nadie sabe por qué. Este error tiene nombre en la industria y es una plaga. Ustedes ya saben cómo se ve — y cómo se previene: la app le pasa al modelo una tabla con nombres, nunca números pelados.

Pasos 4 y 5 · Guardar y Servir

El modelo es un archivo, y Gradio lo vuelve una herramienta

50 – 72 min

El Modelo Es un Archivo

Un modelo entrenado no vive en la sesión de Colab — si se desconecta, se pierde y hay que reentrenar. Se **guarda como archivo**, igual que un documento:

```
import joblib
```

```
joblib.dump(tasador, "tasador_v1.joblib")    # guardar
```

```
tasador = joblib.load("tasador_v1.joblib")   # cargar
```

Ese archivo **es** el producto del entrenamiento. Se versiona (v1, v2...), se comparte, y la app lo carga sin reentrenar nada.

Regla práctica en Colab: guárdenlo también en su Drive. La sesión se apaga; el Drive no.

Qué Es Gradio (lo Pendiente de la Clase 1)

Gradio convierte una función de Python en una página web con controles. Uno le dice «*esta función recibe las medidas y devuelve el precio*», y él arma solo la interfaz: las casillas, el botón, el espacio de la respuesta.

No hay que saber nada de páginas web. Y en Colab, con `share=True`, genera un **link público**: una dirección que el comprador abre desde su celular en la mesa de compra, mientras el cuaderno de ustedes hace el trabajo por detrás.

LO QUE HAY QUE SABER DEL LINK

Vive **mientras el cuaderno esté corriendo**. Si Colab se cierra o se desconecta, el link deja de responder. Perfecto para una demo o una jornada de trabajo; no es un sistema permanente — eso se conversa en la Clase 3.

Gradio en Tres Escalones · 1 y 2

Antes del tasador, la anatomía en chiquito. **Escalón 1** — toda app de Gradio son tres piezas: una función, sus entradas, sus salidas:

```
def saludar(nombre):  
    return f"Hola, {nombre}"
```

```
gr.Interface(fn=saludar, inputs="text", outputs="text").launch()
```

Escalón 2 — controles con tipo y unidades. El mismo patrón ya hace una herramienta de campo:

```
def a_metros_cubicos(barriles):  
    return f"{barriles * 0.158987:,.1f} m3"
```

```
gr.Interface(fn=a_metros_cubicos,  
            inputs=gr.Number(label="Barriles [bbl]", value=1000),  
            outputs=gr.Textbox(label="Metros cúbicos")).launch()
```

Función → **entradas** → **salidas**. El tasador es esto, con un modelo adentro.

Gradio en Tres Escalones · 3: la Salida Es una Gráfica

Escalón 3 — la salida no tiene que ser texto. Con `gr.Plot`, la función devuelve una figura — y la app se vuelve un tablero:

```
def donde_cae(quilates):  
    parecidas = d[abs(d.carat - quilates) < 0.1]  
    fig, ax = plt.subplots(figsize=(7, 3))  
    ax.hist(parecidas.price, bins=40)  
    ax.set_xlabel("precio [USD]")  
    ax.set_title(f"{len(parecidas)} piedras de ~{quilates} quilates")  
    return fig  
  
gr.Interface(fn=donde_cae,  
            inputs=gr.Slider(0.2, 3.0, value=1.0, label="Quilates"),  
            outputs=gr.Plot(label="Precios de piedras similares")).launch()
```

Muevan el deslizador y la distribución cambia: la memoria de comparables, dibujada. **El tasador es el escalón 4.**

El Encargo de la App

CONTEXTO: tengo 'tasador' (RandomForest entrenado), la lista 'columnas' con el orden del entrenamiento, y el DataFrame 'd' certificado.
OBJETIVO: app de Gradio para la mesa de compra – quilates, corte, color, claridad y x, y, z como entrada; el precio estimado en USD como salida.
RESTRICCIONES: controles en español con unidades; corte, color y claridad como desplegables con los valores de 'd'; launch(share=True).
CRITERIO: tasar una piedra en menos de 30 segundos sin ayuda.

Detalle técnico que el agente debe resolver: la app recibe 7 valores, pero el modelo espera las columnas de get_dummies en su orden. Si el agente no lo maneja, ahí está su primer error para cazar.

Paso 6 • Blindar

El tasador funciona. Ahora, que no mienta

72 – 90 min

La Prueba que Nadie le Hace a su Propia App

LO QUE LE PEDIMOS

quilates	0.0
ancho	0 mm
largo	0 mm
profundidad	0 mm

LO QUE RESPONDIÓ

USD 397

*por una piedra que no existe.
Sin dudar. Sin avisar.*

Le pedimos el precio de una piedra de **0 quilates y 0 milímetros** — la misma clase de fila que rechazamos en el paso 1 — y respondió **USD 397**. Sin dudar. Sin avisar.

Por Qué Pasa, y Qué Es un Contrato

El modelo no sabe qué es un diamante. Sabe interpolar entre los casos que vio — y si le dan un punto absurdo, interpola igual y devuelve un número con la misma cara de seguridad.

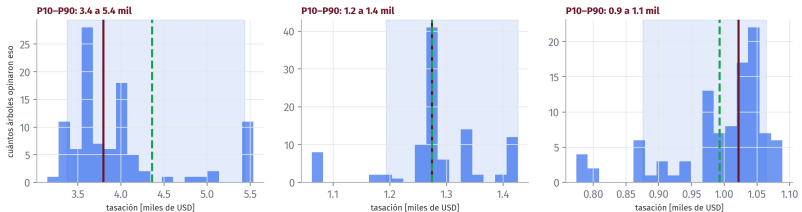
Un modelo servido sin contrato es un generador de basura con interfaz bonita.

El **contrato de entrada y salida** de una herramienta sería:

- **Entrada:** qué valores acepta, y en qué rangos. Los rangos no se inventan: salen **del dato certificado** (mínimos y máximos reales del entrenamiento).
- **Fuera de rango:** la app **no tasa** — explica con un mensaje claro qué está mal.
- **Salida:** no un número seco — un número **con banda**, y con la advertencia si la piedra está cerca del borde de lo que el modelo conoce.

La Banda Sale Gratis: los 100 Árboles No Opinan lo Mismo

Los 100 árboles no opinan lo mismo — y esa discrepancia ES la banda



Cada árbol del bosque da su tasación; la **discrepancia entre ellos es la banda** P10-P90 — la misma idea de las bandas del Módulo 5, ahora dentro de una app. Y miren el panel izquierdo: a veces el precio real queda **fuera** de la banda. Así debe ser: promete el 80 %, no la certeza.

El Encargo del Blindaje

Mejora la app del tasador con un CONTRATO de entrada y salida:

ENTRADA: - calcula los rangos válidos de carat, x, y, z DESDE el DataFrame certificado 'd' (mínimo y máximo) - si el usuario mete un valor fuera de rango, NO tases: muestra un mensaje claro que diga qué campo está mal y cuál es el rango aceptable

SALIDA: - además del precio, la banda P10-P90 calculada con las predicciones de los árboles individuales (tasador.estimators) – *formato* : "USD5.400(banda : 4.900a6.100)"

CRITERIO DE ACEPTACIÓN: - la piedra de 0 mm tiene que ser RECHAZADA con mensaje, no tasada - una piedra normal se tasa igual que antes

El criterio de aceptación es literalmente la prueba que la rompió. Así se blindo: cada error conocido se vuelve una prueba.

Su Turno • La Calculadora de Riesgo

El mismo flujo, de punta a punta, hecho por ustedes

90 – 115 min

El Caso · La Aseguradora Marítima

EL PROBLEMA

Su cliente asegura embarcaciones. Calibra su modelo de riesgo con el naufragio **mejor documentado**: los 891 pasajeros del Titanic.

LA DECISIÓN

Qué factores separaron a quienes sobrevivieron — y cómo pesan en **protocolos** y en la **prima**.

EL COSTO DEL ERROR

Un modelo mal calibrado tarifica mal **todas** las pólizas a la vez.

QUÉ LES PIDIÓ

«La **calculadora de riesgo**: perfil del pasajero, y me da su probabilidad de sobrevivir. Con la seriedad del tasador.»

Es clasificación, no regresión — y el bosque también la sabe hacer.

Desde Cero: Regresión y Clasificación

Hoy vieron la primera; su reto usa la segunda. Es la única diferencia entre la demo y lo que van a hacer:

REGRESIÓN — la demo

La respuesta es un **número**: ¿cuánto vale?

```
RandomForestRegressor  
.predict() → USD 5.400
```

CLASIFICACIÓN — su reto

La respuesta es un **sí o no**: ¿sobrevive?

```
RandomForestClassifier  
.predict_proba() → 0,74
```

Y el consejo profesional: entreguen la probabilidad (`predict_proba`), no el veredicto. «74 de cada 100 como usted sobrevivieron» le sirve a una aseguradora; un «sí» seco, no. *Todo lo demás — certificar, reservar el examen, guardar, servir, blindar — es idéntico. Por eso el flujo es lo que se llevan, no el modelo.*

🔧 Su Reto: los Mismos Seis Pasos

🕒 25 min

El flujo completo, con sus propios prompts (la plantilla de la C1 sirve en cada paso):

- 1 Certificar:** faltantes, ceros, columnas sospechosas.
- 2 Entrenar:** RandomForestClassifier, 20 % reservado antes.
- 3 Examinar:** de cada 100 del examen, ¿a cuántos acierta?
- 4 Guardar** con joblib. **5 Servir:** Gradio con predict_proba.
- 6 Blindar:** rangos desde el dato; edad de 500 años, rechazada con mensaje.

UNA ADVERTENCIA, SIN MÁS PISTAS

Si su modelo acierta **casi todo**, no celebren: **paren y desconfíen**. Algo en sus columnas huele — encuéntralo.

Puesta en Común

Entre mesas, cuatro preguntas:

- 🗨️ Al certificar: ¿qué encontraron — cuántos faltantes, en qué columnas? ¿Qué decidieron hacer con la edad?
- 🗨️ ¿Qué columnas le dieron al modelo, y **quién** eligió: ustedes en el prompt, o el agente por defecto?
- 🗨️ ¿Qué acierto sacaron? ¿Alguna mesa sacó **casi 100 de 100**? — que nos cuente qué columnas usó, y lo miramos juntos.
- 🗨️ La calculadora: ¿devuelve un veredicto («sobrevive») o una **probabilidad**? ¿Cuál de las dos le sirve a una aseguradora?

La tercera pregunta es la importante. Si nadie cayó, lo provocamos juntos — porque en el trabajo real, esa trampa la van a pisar.

Cierre

Lo Que Aprendimos Hoy

💡 Conceptos:

- ✓ El **flujo completo**: certificar, entrenar, examinar, guardar, servir, blindar
- ✓ El examen con datos **reservados antes** de entrenar
- ✓ El error **en la unidad del negocio**: dólares, no R^2
- ✓ El **contrato** de entrada y salida
- ✓ La **banda** que sale de los árboles en desacuerdo

🔧 Herramientas:

- ➔ RandomForestRegressor / Classifier
- ➔ train_test_split — el 20 % reservado
- ➔ joblib — el modelo como archivo
- ➔ gradio con share=True — por fin, el link

EL NÚMERO DEL DÍA

USD 397. Lo que el tasador sin contrato pagó por una piedra que no existe.

Los Resultados Negativos de Hoy

Como siempre, en voz alta:

- ✗ **El tasador tasó una piedra de 0 mm en USD 397**, con total confianza. Un modelo no sabe cuándo la pregunta es absurda; eso lo tiene que saber la app — y la app la escriben ustedes.
- ✗ **La banda a veces no atrapa el precio real** — lo vimos en la propia lámina. Promete el 80 %, y cumple el 80 %: la certeza no existe, y venderla sería mentir.
- ✗ Y el que hayan encontrado ustedes en el reto — el modelo **demasiado bueno**. Lo que huele a perfecto, huele a trampa.

Los tres son la misma lección: el modelo hace números; el criterio de cuándo creerle sigue siendo de ustedes.

Si Se Llevan Una Sola Cosa

Un modelo no es el producto. El producto es la herramienta que lo envuelve: certificada, examinada y blindada.

El modelo se entrenó en 2 segundos. Todo lo demás de la clase — certificar el dato, reservar el examen, el contrato, la banda, los mensajes — es lo que separa un juguete de una herramienta que alguien puede usar para decidir con plata.

Y noten la proporción: 2 segundos de modelo, 2 horas de ingeniería alrededor. Esa proporción es la real en la industria.

La Lista que se Llevan: los Seis Pasos, para SU Proyecto

Imprimible. Cada paso, con la pregunta que lo aprueba:

1. **Certificar** — ¿revisé ceros, faltantes e imposibles físicos, y escribí qué rechacé y por qué?
2. **Entrenar** — ¿reservé el examen *antes*? ¿elegí yo las columnas, o las eligió el agente sin decirme?
3. **Examinar** — ¿tengo el error en la *unidad del negocio*? ¿se lo puedo decir a mi jefe en una frase?
4. **Guardar** — ¿el modelo es un archivo con versión, o vive en una sesión de Colab que se va a apagar?
5. **Servir** — ¿alguien que no soy yo puede usarlo en 30 segundos, desde un link?
6. **Blindar** — ¿la entrada absurda es rechazada con mensaje? ¿la salida lleva banda?

Seis preguntas con sí: proyecto entregable. Cualquier no: ya saben exactamente qué falta.

Su Proyecto: Hoy Quedó la Pieza Central

Lo que hicieron hoy con el Titanic es **exactamente** lo que su proyecto necesita con su propio dato:

- ✓ El flujo de seis pasos, con sus prompts
- ✓ La app con link — el entregable
- ✓ Y seguramente ya cazaron **un error del agente** — anótenlo, que es parte de la entrega

✓ PARA ESTA SEMANA

Entrega hasta el **jueves de la próxima semana**, equipos de hasta 3. En la Clase 3: los peligros con nombre — qué se puede pegar en un chat de IA y qué no, antes de que trabajen con datos de la empresa.

Datos y Referencias

-  **Datos:** diamantes (la demo) y pasajeros_titanic (el reto), publicados en el repositorio del curso — copias exactas de los conjuntos de práctica de seaborn.
-  **scikit-learn, joblib y Gradio:** las tres vienen instaladas en Colab.
-  Las figuras y **todas** las cifras de estas láminas salen de `figuras.py` con semilla fija. Las soluciones del reto **no existen en ningún archivo:** se construyen en clase.

Como siempre: corren el script y tiene que dar exactamente lo mismo. Si no da, es un error nuestro y queremos saberlo.

¡Gracias!

Carlos Enrique Mosquera Trujillo

✉ cmosquerat@unal.edu.co

Machine Learning for Petroleum Engineers Using Python

Módulo 7 · Clase 2 · SLB Ecuador · UDLA · 2026